

Pre-entrega 6

Analítica avanzada in-stream con Amazon Redshift

Streaming Ingestion + Redshift Serverless + Apache Iceberg

Proyecto: realtime-data-platform

Alumno: Marcos Pim

Curso: Data Engineering

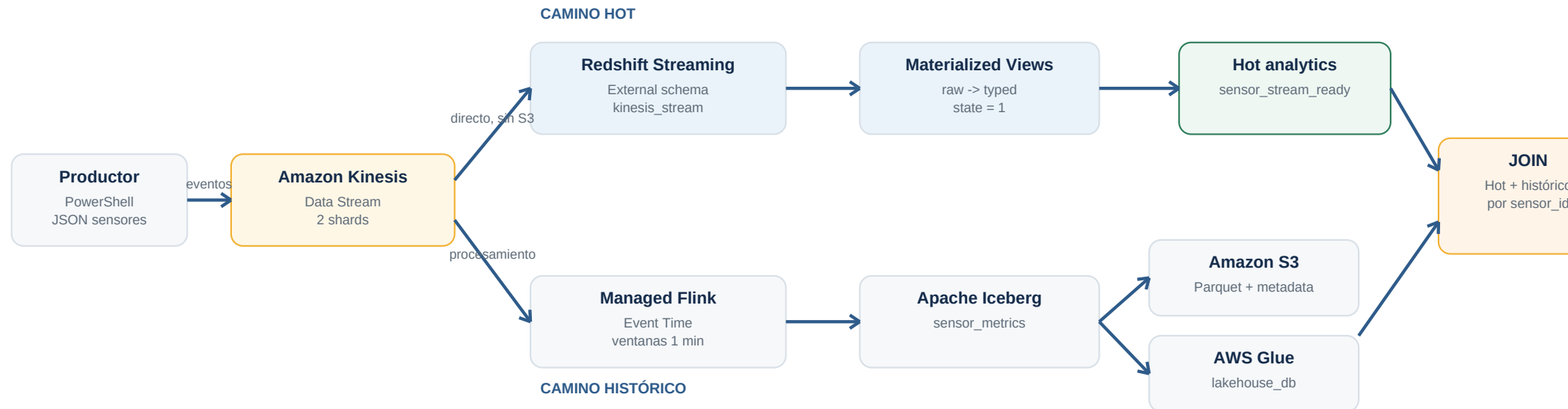
Fecha: 26 de agosto de 2026

Terraform + AWS

1. Objetivo y arquitectura

Extensión de la plataforma para análisis de baja latencia sin perder la capa histórica Lakehouse.

La sexta etapa agrega **Amazon Redshift Serverless** como capa analítica. Los mismos eventos siguen dos caminos complementarios: un camino **hot**, consumido directamente desde Amazon Kinesis mediante Redshift Streaming Ingestion, y un camino **histórico**, procesado por Apache Flink y persistido como Apache Iceberg en Amazon S3 con AWS Glue Data Catalog.



Resultado: Redshift puede consultar datos calientes y datos históricos dentro de la misma sesión SQL y combinarlos por sensor_id.

La arquitectura conserva el Lakehouse existente y agrega una ruta hot directa a Redshift, evitando duplicar la lógica de ingesta.

2. Infraestructura, red e IAM

Recursos declarados con Terraform para Redshift Serverless y conectividad privada.

Infraestructura de Redshift

- **Namespace:** realtime-data-platform-dev.
- **Workgroup:** realtime-data-platform-dev-wg.
- **Base / max capacity:** 4 RPU.
- **Base de datos:** analytics.
- **Acceso público:** deshabilitado.
- **Usage limit:** 8 RPU-hours diarios, acción deactivar.

Red privada

- Tercera subred privada: 10.0.3.0/24 en us-east-1c.
- `enable_dns_support = true` y `enable_dns_hostnames = true`.
- Interface VPC Endpoint para `kinesis-streams` con Private DNS.

Elemento	Propósito	Resultado validado
VPC / 3 subredes	Alojamiento privado del Workgroup	Redshift Serverless AVAILABLE
Kinesis Interface Endpoint	Acceso privado al stream	Private DNS habilitado
Security Groups	HTTPS 443 entre Redshift y endpoint	Conectividad funcional
IAM role Redshift	Kinesis + Glue + S3 + KMS	Streaming + Spectrum/Iceberg funcional

Rol de servicio de Redshift

El módulo crea un rol IAM específico para que Redshift consuma Kinesis y lea el catálogo y los objetos del Lakehouse. Los permisos de servicio están separados de los permisos administrativos utilizados durante el laboratorio.

```
kinesis:DescribeStream
kinesis:DescribeStreamSummary
kinesis:GetShardIterator
kinesis:GetRecords
kinesis:ListShards
kinesis:ListStreams
```

También se habilitan permisos de lectura para AWS Glue y Amazon S3, más `kms:Decrypt` condicionado al uso a través del servicio Kinesis.

Principio aplicado

Separación de responsabilidades: Terraform administra infraestructura e IAM de servicios; el script SQL versionado administra objetos analíticos de Redshift.

3. Streaming Ingestion: Kinesis -> Redshift

Ingesta directa de eventos sin utilizar Amazon S3 como intermediario.

External Schema de Kinesis

```
CREATE EXTERNAL SCHEMA kinesis_stream
FROM KINESIS
IAM_ROLE 'arn:aws:iam::<account-id>:role/realtime-data-
platform-dev-redshift-role';
```

Materialized View raw

La vista sensor_stream_raw consume directamente el stream y conserva metadata de Kinesis, el payload parseado y un campo para eventos inválidos.

```
CASE WHEN CAN_JSON_PARSE(kinesis_data)
THEN JSON_PARSE(kinesis_data)
ELSE NULL END AS payload
```

En la evidencia, failed_payload permanece en NULL, indicando que los eventos mostrados pudieron interpretarse correctamente como JSON.

Redshift query editor v2

Filter resources

Serverless: realtime-data-...

native databases (5)

external databases (1)

Run

Limit 100

Explain

Isolated session

Serverless: re...

analytics

```
1 SELECT
2   approximate_arrival_timestamp,
3   partition_key,
4   shard_id,
5   payload,
6   failed_payload
7 FROM sensor_stream_raw
8 ORDER BY approximate_arrival_timestamp DESC
9 LIMIT 20;
```

Result 1 (20)

approximate_arrival...	partition_key	shard_id	payload	failed_payload
2026-08-26 16:16:03.717	sensor-01	shardId-0000000000001	("sensor_id":"sensor-01",...	NULL
2026-08-26 16:16:02.102	sensor-05	shardId-0000000000000	("sensor_id":"sensor-05",...	NULL
2026-08-26 16:16:00.493	sensor-04	shardId-0000000000001	("sensor_id":"sensor-04",...	NULL
2026-08-26 16:15:58.766	sensor-03	shardId-0000000000001	("sensor_id":"sensor-03",...	NULL
2026-08-26 16:15:57.03	sensor-02	shardId-0000000000000	("sensor_id":"sensor-02",...	NULL
2026-08-26 16:15:55.353	sensor-01	shardId-0000000000001	("sensor_id":"sensor-01",...	NULL
2026-08-26 16:15:53.672	sensor-05	shardId-0000000000000	("sensor_id":"sensor-05",...	NULL
2026-08-26 16:15:51.99	sensor-04	shardId-0000000000001	("sensor_id":"sensor-04",...	NULL
2026-08-26 16:15:50.038	sensor-03	shardId-0000000000001	("sensor_id":"sensor-03",...	NULL
2026-08-26 16:15:48.245	sensor-02	shardId-0000000000000	("sensor_id":"sensor-02",...	NULL
2026-08-26 16:15:46.41	sensor-01	shardId-0000000000001	("sensor_id":"sensor-01",...	NULL
2026-08-26 16:15:44.30	sensor-05	shardId-0000000000000	("sensor_id":"sensor-05",...	NULL
2026-08-26 16:15:42.452	sensor-04	shardId-0000000000001	("sensor_id":"sensor-04",...	NULL
2026-08-26 16:15:40.509	sensor-03	shardId-0000000000001	("sensor_id":"sensor-03",...	NULL
2026-08-26 16:15:38.804	sensor-02	shardId-0000000000000	("sensor_id":"sensor-02",...	NULL
2026-08-26 16:15:37.163	sensor-01	shardId-0000000000001	("sensor_id":"sensor-01",...	NULL

Evidencia: eventos consumidos por sensor_stream_raw desde Kinesis. Se observan ambos shards, partition keys y payloads JSON válidos.

Campos del evento: sensor_id, timestamp, temperature, humidity y air_quality_index.

4. Modelado SQL y mantenimiento incremental

Separación entre ingestión raw, tipado incremental y vista final lista para consumo.

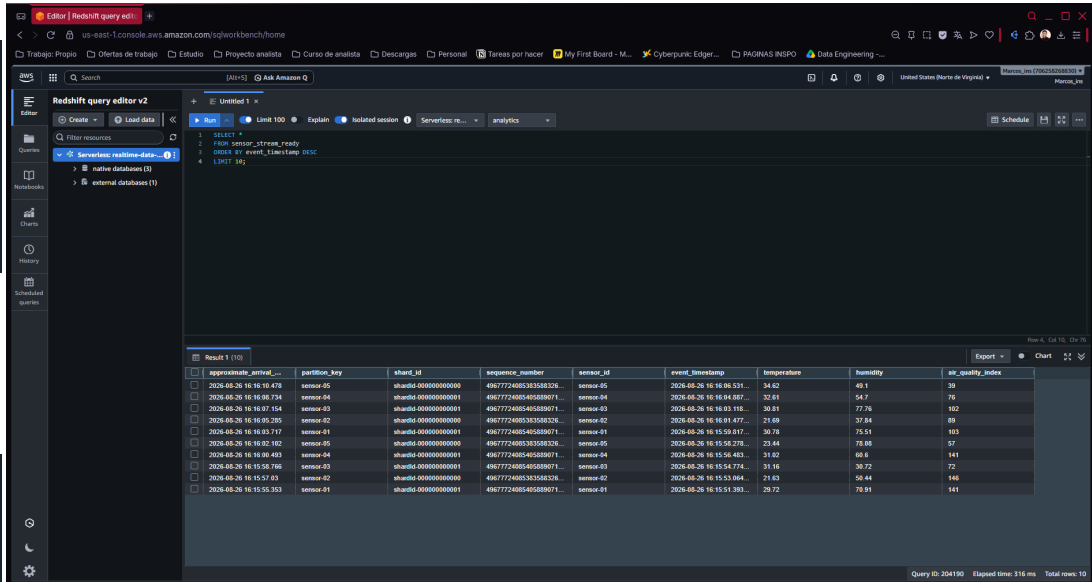
sensor_stream_typed

```
SELECT
payload.sensor_id::VARCHAR AS sensor_id,
payload.timestamp::VARCHAR AS event_timestamp_raw,
payload.temperature::DECIMAL(5,2) AS temperature,
payload.humidity::DECIMAL(5,2) AS humidity,
payload.air_quality_index::INTEGER AS air_quality_index
FROM sensor_stream_raw
WHERE payload IS NOT NULL;
```

El timestamp se mantiene como VARCHAR en la MV porque la conversión con TRY_CAST(... AS TIMESTAMPTZ) impedía el mantenimiento incremental.

sensor_stream_ready

```
CREATE OR REPLACE VIEW sensor_stream_ready AS
SELECT ...,
TRY_CAST(event_timestamp_raw AS TIMESTAMPTZ) AS event_timestamp,
temperature, humidity, air_quality_index
FROM sensor_stream_typed;
```



Evidencia: sensor_stream_ready expone timestamps y métricas tipadas para consumo analítico.

MV	state	autorefresh	Estrategia
sensor_stream_raw	1	false	Refresh manual durante desarrollo
sensor_stream_typed	1	false	Refresh manual después de raw

Interpretación: state = 1 valida mantenimiento incremental. autorefresh = false fue una decisión de desarrollo para controlar costo y momento de procesamiento; en producción podría usarse autorefresh o una frecuencia de 30-60 s según SLA.

5. Integración con Apache Iceberg

Redshift consulta el histórico mediante AWS Glue Data Catalog y los objetos Iceberg almacenados en S3.

External Schema del Lakehouse

```
CREATE EXTERNAL SCHEMA lakehouse_ext
FROM DATA CATALOG
DATABASE 'lakehouse_db'
REGION 'us-east-1'
IAM_ROLE 'arn:aws:iam::<account-id>:role/realtime-data-platform-dev-redshift-role';
```

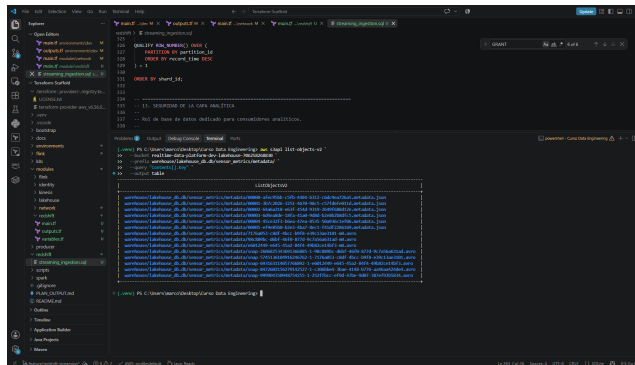
La tabla histórica es lakehouse_ext.sensor_metrics. Cada fila corresponde a una ventana de un minuto procesada por Flink.

Esquema histórico

```
sensor_id
window_start
window_end
event_count
avg_temperature
avg_humidity
avg_aqi
```

sensor_id	window_start	window_end	event_count	avg_temperature	avg_humidity	avg_aqi
sensor-01	2026-08-26 16:57:00	2026-08-26 16:58:00	9	25.307777777777776	62.587777777777774	75.88888888888889
sensor-02	2026-08-26 16:57:00	2026-08-26 16:58:00	9	23.13111111111111	65.38666666666667	96.77777777777777
sensor-05	2026-08-26 16:57:00	2026-08-26 16:58:00	9	26.484444444444447	68.42533333333333	87.55555555555556
sensor-03	2026-08-26 16:57:00	2026-08-26 16:58:00	8	25.893750000000004	56.4075	83.825
sensor-04	2026-08-26 16:57:00	2026-08-26 16:58:00	8	27.333750000000002	63.86375000000001	69.375
sensor-03	2026-08-26 16:56:00	2026-08-26 16:57:00	9	25.987777777777778	62.895555555555556	79.11111111111111
sensor-02	2026-08-26 16:56:00	2026-08-26 16:57:00	9	23.142222222222223	50.089999999999996	107.88888888888889
sensor-04	2026-08-26 16:56:00	2026-08-26 16:57:00	9	25.36111111111111	62.99111111111111	81.66666666666667
sensor-05	2026-08-26 16:56:00	2026-08-26 16:57:00	8	24.97875	58.7975	79.125
sensor-01	2026-08-26 16:56:00	2026-08-26 16:57:00	8	22.551250000000003	56.4075	77.125

Evidencia: consulta de lakehouse_ext.sensor_metrics desde Redshift Query Editor v2.



Metadata Iceberg validada en S3.

Se observaron múltiples versiones metadata.json, manifests .avro y snapshots snap-*.avro dentro de warehouse/lakehouse_db.db/sensor_metrics/metadata/.

6. JOIN entre datos hot e históricos

Validación principal: correlación del último evento de cada sensor con su última ventana histórica Iceberg.

Se utilizó ROW_NUMBER() para seleccionar el evento hot más reciente de cada sensor y, en paralelo, la ventana Iceberg más reciente. Luego ambos conjuntos se unieron por sensor_id.

```
WITH latest_hot AS (  
  SELECT ..., ROW_NUMBER() OVER (PARTITION BY sensor_id ORDER BY event_timestamp DESC) rn  
  FROM sensor_stream_ready  
)  
,  
latest_historical AS (  
  SELECT ..., ROW_NUMBER() OVER (PARTITION BY sensor_id ORDER BY window_end DESC) rn  
  FROM lakehouse_ext.sensor_metrics  
)  
SELECT ...  
FROM latest_hot h  
JOIN latest_historical i ON h.sensor_id = i.sensor_id  
WHERE h.rn = 1 AND i.rn = 1;
```

sensor id	hot event timestamp	hot temperature	hot humidity	hot aqi	historical window st...	historical window end	event count	historical avg tempe...	historical avg humi...	historical avg aqi
sensor-01	2026-08-26 16:58:05.360...	24.2	87.38	54	2026-08-26 16:57:00	2026-08-26 16:58:00	9	25.307777777777776	62.587777777777774	75.8888888888889
sensor-02	2026-08-26 16:58:06.714...	31.38	88.94	71	2026-08-26 16:57:00	2026-08-26 16:58:00	9	23.131111111111111	65.38666666666667	96.77777777777777
sensor-03	2026-08-26 16:58:08.091...	30.6	32.67	120	2026-08-26 16:57:00	2026-08-26 16:58:00	8	25.893750000000004	56.4075	83.625
sensor-04	2026-08-26 16:58:09.553...	30.97	75.76	68	2026-08-26 16:57:00	2026-08-26 16:58:00	8	27.333750000000002	63.66375000000001	69.375
sensor-05	2026-08-26 16:58:10.910...	34.14	64.6	25	2026-08-26 16:57:00	2026-08-26 16:58:00	9	20.494444444444447	68.42333333333335	87.55555555555556

Evidencia principal: cinco sensores con métricas hot (evento individual) y métricas históricas (ventana Iceberg) en una misma consulta SQL

Resultado: se validó el patrón híbrido de consulta requerido: baja latencia para datos recién llegados y contexto histórico desde el Lakehouse.

7. Monitoreo y seguridad

Observabilidad del stream y separación de privilegios para consumidores analíticos.

Monitoreo de Streaming Ingestion

La vista SYS_STREAM_SCAN_STATES se usó para inspeccionar el último scan por shard, filas escaneadas, filas omitidas y lag observado.

```
DATEDIFF(second, stream_record_time_max, record_time)
AS ingestion_lag_seconds
```

Valores observados en una ejecución de prueba:

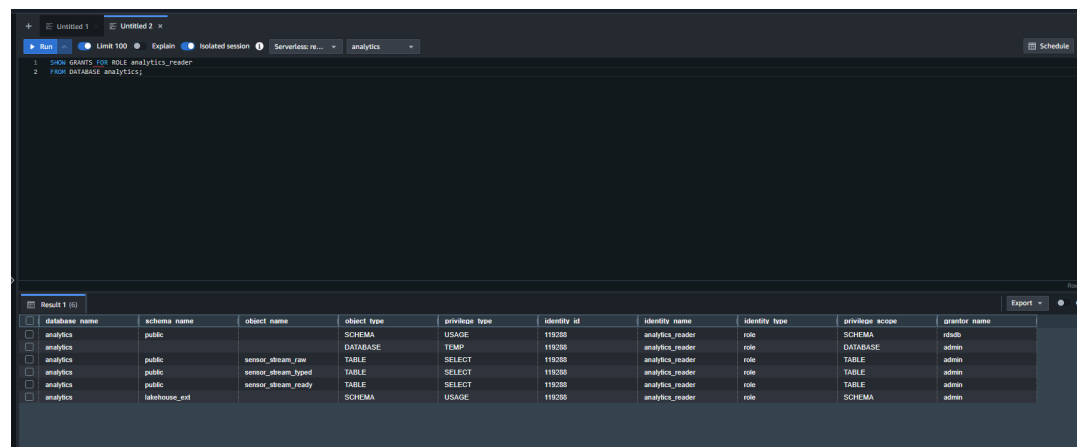
Shard	Scanned	Skipped	Lag observado
0	26	0	311 s
1	33	0	313 s

El lag incluye el tiempo transcurrido antes del refresh manual; por eso no representa la latencia mínima posible del servicio. **skipped_rows = 0** confirma que no se omitieron registros en esos scans.

Rol SQL analytics_reader

Se creó un rol de base de datos separado de la identidad administradora para restringir la capa analítica.

```
GRANT TEMP ON DATABASE analytics TO ROLE analytics_reader;
GRANT USAGE ON SCHEMA public TO ROLE analytics_reader;
GRANT SELECT ON public.sensor_stream_raw TO ROLE analytics_reader;
GRANT SELECT ON public.sensor_stream_typed TO ROLE analytics_reader;
GRANT SELECT ON public.sensor_stream_ready TO ROLE analytics_reader;
GRANT USAGE ON SCHEMA lakehouse_ext TO ROLE analytics_reader;
```



Evidencia: grants efectivos del rol analytics_reader sobre la base analytics, el schema public, las vistas hot y el External Schema del Lakehouse.

8. Conclusiones y entregables

Validación end-to-end de la Pre-entrega 6.

Resultados validados

- **Ingesta directa:** Kinesis -> Redshift Streaming Ingestion sin S3 intermedio.
- **JSON:** validación con CAN_JSON_PARSE, parseo a SUPER y tipado SQL.
- **Materialized Views:** raw y typed con mantenimiento incremental validado (state = 1).
- **Lakehouse:** consulta de sensor_metrics vía AWS Glue Data Catalog y metadata Iceberg presente en S3.
- **Analítica híbrida:** JOIN exitoso entre eventos hot y agregados históricos por sensor_id.
- **Observabilidad:** lag por shard y skipped_rows = 0 mediante SYS_STREAM_SCAN_STATES.
- **Seguridad:** rol SQL analytics_reader con privilegios restringidos.
- **Infraestructura:** Redshift Serverless privado, 4 RPU, VPC Endpoint de Kinesis y límite diario de consumo.

Criterio	Estado
Redshift Streaming Ingestion desde Kinesis	VALIDADO
Materialized Views y parseo >= 3 campos	VALIDADO
External Schema Glue / Iceberg	VALIDADO
JOIN hot + histórico	VALIDADO
Refresh incremental / estrategia documentada	VALIDADO
Rol analítico restringido	VALIDADO
Monitoreo de lag	VALIDADO
Evidencias y SQL consolidado	VALIDADO

Artefactos versionados

```
modules/redshift/main.tf
modules/redshift/variables.tf
modules/redshift/outputs.tf
redshift/streaming_ingestion.sql
docs/evidencia-redshift-*.png
README.md
```

Estrategia de desarrollo

La infraestructura se despliega con Terraform y los objetos SQL se mantienen en un script consolidado. Flink se detiene cuando no se utiliza y Redshift Serverless posee un límite diario de compute para reducir costos durante el laboratorio.

Limpieza

```
cd environments\dev
terraform destroy
```